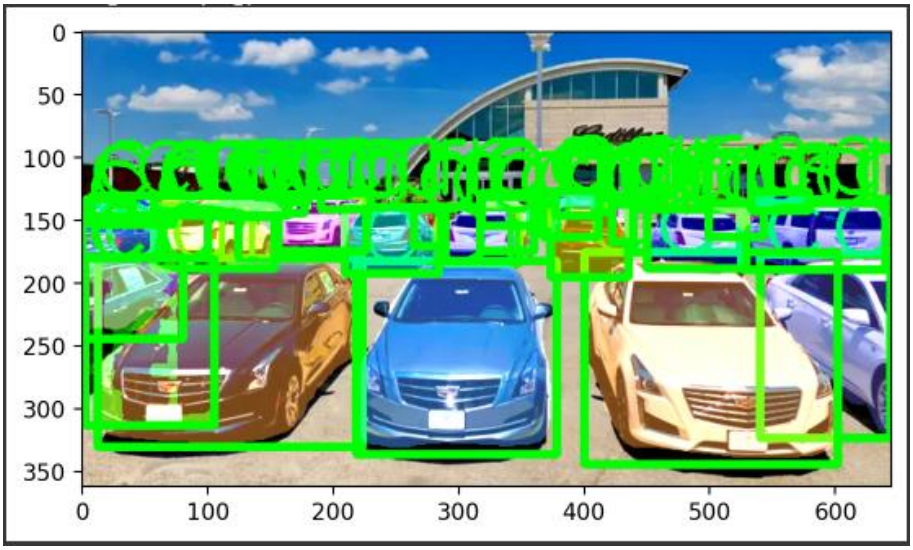
RCNN:

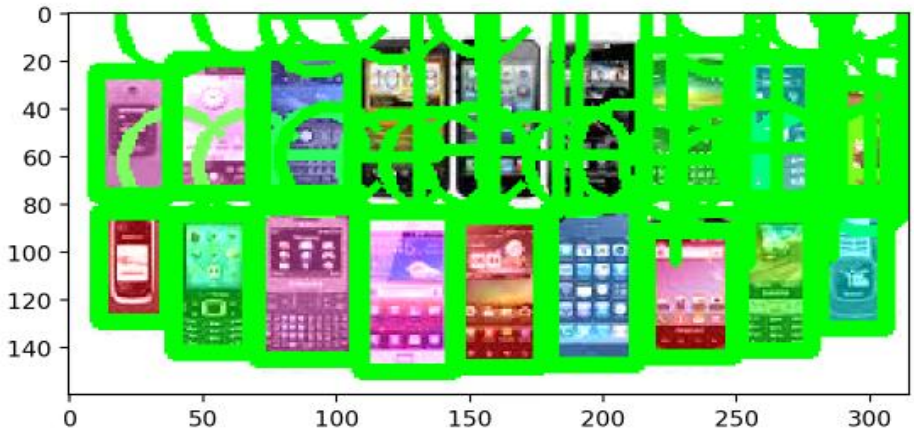
```
import torch
import torchvision
from PIL import Image
from torchvision import transforms as T
import numpy as np
import requests
from io import BytesIO
import matplotlib.pyplot as plt
import cv2
from urllib.request import urlopen

# Load pre-trained Mask R-CNN model
model =
torchvision.models.detection.maskrcnn_resnet50_fpn(pretrained=True)
model.eval()
model = model.cuda()

# COCO class names
COCO_INSTANCE_CATEGORY_NAMES = [
    '__background__', 'person', 'bicycle', 'car', 'motorcycle',
'airplane', 'bus',
    'train', 'truck', 'boat', 'traffic light', 'fire hydrant', 'N/A',
'stop sign',
    'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse', 'sheep',
'cow',
    'elephant', 'bear', 'zebra', 'giraffe', 'N/A', 'backpack', 'umbrella',
'N/A', 'N/A',
    'handbag', 'tie', 'suitcase', 'frisbee', 'skis', 'snowboard', 'sports
ball',
    'kite', 'baseball bat', 'baseball glove', 'skateboard', 'surfboard',
'tennis racket',
    'bottle', 'N/A', 'wine glass', 'cup', 'fork', 'knife', 'spoon',
'bowl',
    'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot', 'hot
dog', 'pizza',
    'donut', 'cake', 'chair', 'couch', 'potted plant', 'bed', 'N/A',
'dining table',
    'N/A', 'N/A', 'toilet', 'N/A', 'tv', 'laptop', 'mouse', 'remote',
'keyboard', 'cell phone',
    'microwave', 'oven', 'toaster', 'sink', 'refrigerator', 'N/A', 'book',
    'clock', 'vase', 'scissors', 'teddy bear', 'hair drier', 'toothbrush'
```

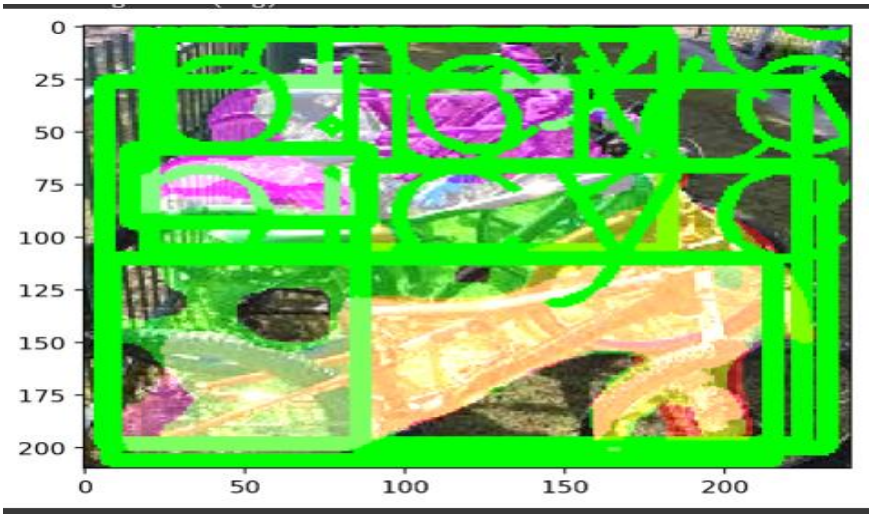
```
]

def get_prediction(img_path, threshold=0.5, url=False):
    if url:
        response = requests.get(img_path)
        img = Image.open(BytesIO(response.content))
    else:
        img = Image.open(img_path)
    transform = T.Compose([T.ToTensor()])
    img = transform(img)
    img = img.cuda()
    pred = model([img])
    pred_score = list(pred[0]['scores'].detach().cpu().numpy())
    pred_t = [pred_score.index(x) for x in pred_score if x > threshold][-
1]
    masks = (pred[0]['masks'] > 0.5).squeeze().detach().cpu().numpy()
    pred_class = [COCO_INSTANCE_CATEGORY_NAMES[i] for i in
list(pred[0]['labels'].cpu().numpy())]
    pred_boxes = [[(i[0], i[1]), (i[2], i[3])] for i in
list(pred[0]['boxes'].detach().cpu().numpy())]
    masks = masks[:pred_t + 1]
    pred_boxes = pred_boxes[:pred_t + 1]
    pred_class = pred_class[:pred_t + 1]
    return masks, pred_boxes, pred_class

def url_to_image(url, readFlag=cv2.IMREAD_COLOR):
    resp = urlopen(url)
    image = np.asarray(bytearray(resp.read()), dtype="uint8")
    image = cv2.imdecode(image, readFlag)
    return image

def random_color_masks(image):
    colors = [[0, 255, 0],[0, 0, 255],[255, 0, 0],[0, 255, 255],[255, 255,
0],[255, 0, 255],[80, 70, 180], [250, 80, 190],[245, 145, 50],[70, 150,
250],[50, 190, 190]]
    r = np.zeros_like(image).astype(np.uint8)
    g = np.zeros_like(image).astype(np.uint8)
    b = np.zeros_like(image).astype(np.uint8)
    r[image == 1], g[image == 1], b[image == 1] =
colors[random.randrange(0, 10)]
    colored_mask = np.stack([r, g, b], axis=2)
    return colored_mask

def instance_segmentation(img_path, threshold=0.5, rect_th=3, text_size=3,
text_th=3, url=False):
```

```
    masks, boxes, pred_cls = get_prediction(img_path, threshold=threshold,
url=url)
    if url:
        img = url_to_image(img_path)
    else:
        img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    for i in range(len(masks)):
        rgb_mask = random_color_masks(masks[i])
        img = cv2.addWeighted(img, 1, rgb_mask, 0.5, 0)
        pt1 = tuple(int(x) for x in boxes[i][0])
        pt2 = tuple(int(x) for x in boxes[i][1])
        cv2.rectangle(img, pt1, pt2, color=(0, 255, 0), thickness=rect_th)
        cv2.putText(img, pred_cls[i], pt1, cv2.FONT_HERSHEY_SIMPLEX,
text_size, (0, 255, 0), thickness=text_th)
    return img, pred_cls, masks[i]

# Assuming the image is uploaded and named "uploaded_image.jpg"
uploaded_image_path = "/content/qaek94apoucbe5fcrmbw.webp"

# Open the uploaded image
uploaded_image = Image.open(uploaded_image_path)

# Display the image
uploaded_image.show()

# Run instance segmentation
img, pred_classes, masks =
instance_segmentation('/content/qaek94apoucbe5fcrmbw.webp', rect_th=5,
text_th=4)

# Display the segmented image
plt.imshow(img)
plt.show()
```

OUTPUTS:


0
50
100
150
200
250
300
350
0
100
200
300
400
500
600


0
20
40
60
80
100
120
140
0
50
100
150
200
250
300


0
25
50
75
100
125
150
175
200
0
50
100
150
200

0
100
200
300
400
0
100
200
300
400
500
600